Analysis of Inventory Performance

1. contains() Performance
   - Hash Inventory: Achieves $O(1)$ average time complexity because std::unordered_set hashes items directly by name, inspecting the target bucket without scanning other elements.
   - Tree Inventory (ItemAVL): Scales linearly ($O(N)$), increasing from 0.0049 ms ($N = 1000$) to 0.0852 ms ($N = 8000$). Because the tree is ordered by a custom Comparator (e.g., weight), searching by name requires a full recursive traversal across all nodes.
   - Vector & List Inventories: Require an $O(N)$ linear scan over elements, though simple layout keeps absolute runtimes small at these test sizes.

2. query() Performance
   - Vector, List & Hash Inventories: Require an $O(N)$ full traversal to check every element against the $[start, end]$ range. At $N = 8000$ for weight queries, Hash takes 0.1538 ms, List takes 0.0798 ms, and Vector takes 0.0713 ms.
   - Tree Inventory (ItemAVL): Leverages its binary search tree structure to prune subtrees outside range bounds in queryHelper(). This optimizes range retrieval to $O(\log N + K)$ time, completing in just 0.0104 ms at $N = 8000$.

3. Container Selection Guidelines
   - Choose Hash-based Inventory: When primary workloads consist of single-item lookups, insertions, or deletions by name in $O(1)$ time. Range queries are inefficient due to unordered elements.
   - Choose Tree-based Inventory: When primary workloads require frequent range queries on ordered attributes or sorted traversals. Tree pruning drastically reduces range search time, though searching by unindexed attributes degrades to $O(N)$.